

Library Management System

Project Objective

Design and implement a **console-based Library Management System in Java** that enables a library to maintain its collection of books, register members, issue and return books, and manage the availability of books. The application should be developed using object-oriented programming principles and should provide appropriate validation and exception handling for invalid operations.

Problem Description

A library maintains a collection of books belonging to different categories. Each book has common information such as a unique book ID, title, author, price, and availability status. The library may contain different types of books such as **Printed Books, Reference Books, and E-Books**. While these books share common characteristics, their behavior may differ depending upon their type.

The system should allow the librarian to add books to the library and register library members. Each member should have a unique member ID, name, email address, and details of the books currently issued to them.

A registered member should be able to search for a book using its ID, issue an available book, return an issued book, and view the details of their issued books. The system must ensure that a book cannot be issued if it is already issued to another member.

The application should contain a base **Book** class representing the common properties and behavior of books. Specialized classes such as **PrintedBook**, **ReferenceBook**, and **EBook** should represent different types of books and may implement their specific behavior. The application should demonstrate inheritance and polymorphism wherever appropriate.

A **Library** class should manage the complete library inventory and registered members. It should provide functionality for adding books, registering members, searching for books, issuing books, returning books, and displaying available books.

The system should also implement appropriate validation and exception handling. For example, an appropriate error should be reported when a user attempts to issue an unavailable book, searches for a non-existing book, provides an invalid member ID, or attempts to return a book that was not issued.

The complete system should be executed through a launcher/main class that demonstrates the different operations and their expected results.

Functional Requirements

1. Add Book

The system should allow a new book to be added with:

- Book ID
- Book title
- Author
- Price
- Book type

The book should initially be marked as **Available**.

2. Register Member

The system should allow a new library member to be registered with:

- Member ID
- Member name
- Email

3. Search Book

The system should search for a book using its unique Book ID.

4. Issue Book

A registered member should be able to issue an available book.

After successful issue:

- The book status should become *Issued*.
- The book should be associated with the member.

5. Return Book

A member should be able to return a book previously issued to them.

After successful return:

- The book status should become *Available*.
- The book should be removed from the member's issued-book list.

6. Display Available Books

The system should display all books whose current status is `Available`.

7. Display Member Details

The system should display the member's details along with the books currently issued to the member.

Required OOP Structure

The implementation should appropriately use the following concepts:

Book

Base class containing common book information and operations.

PrintedBook

Derived class representing a physical printed book.

ReferenceBook

Derived class representing a reference book.

EBook

Derived class representing an electronic book.

Member

Represents a library member.

Library

Manages books, members, and library operations.

Exception Handling

Appropriate exceptions should be used for invalid operations such as:

- Book not found
- Member not found
- Book unavailable

- Invalid return operation
-

Input / Output Test Cases

The following test cases can be used to validate the project. The assessment engine can invoke the corresponding methods with the specified inputs and compare the returned/displayed result with the expected output.

Test Case 1 – Add a Valid Book

Input

```
Book ID: 101
Title: Java Programming
Author: James
Price: 500
Type: PrintedBook
```

Expected Output

```
Book added successfully
Book ID: 101
Title: Java Programming
Author: James
Price: 500.0
Status: Available
```

Test Case 2 – Register a Valid Member

Input

```
Member ID: M101
Name: Amit
Email: amit@gmail.com
```

Expected Output

```
Member registered successfully
Member ID: M101
Name: Amit
Email: amit@gmail.com
```

Test Case 3 – Search Existing Book

Input

Book ID: 101

Expected Output

Book Found
Book ID: 101
Title: Java Programming
Author: James
Price: 500.0
Status: Available

Test Case 4 – Search Non-Existing Book

Input

Book ID: 999

Expected Output

Book Not Found

The implementation may use a custom `BookNotFoundException` or an equivalent defined exception-handling mechanism.

Test Case 5 – Issue Available Book

Input

Member ID: M101
Book ID: 101

Expected Output

Book issued successfully
Member ID: M101
Book ID: 101
Status: Issued

Test Case 6 – Verify Book Status After Issue

Input

Book ID: 101

Expected Output

Book Found
Book ID: 101
Title: Java Programming
Status: Issued

This test verifies that issuing the book changes its availability status.

Test Case 7 – Attempt to Issue an Already Issued Book

Input

Member ID: M102
Book ID: 101

Expected Output

Book is not available

The system should not allow the same book to be issued to another member.

Test Case 8 – Return an Issued Book

Input

Member ID: M101
Book ID: 101

Expected Output

Book returned successfully
Book ID: 101
Status: Available

Test Case 9 – Verify Book Status After Return

Input

Book ID: 101

Expected Output

```
Book Found  
Book ID: 101  
Title: Java Programming  
Status: Available
```

Test Case 10 – Return a Book That Was Not Issued

Input

```
Member ID: M101  
Book ID: 101
```

Expected Output

```
Book was not issued to this member
```

This test should be performed when the book is already available.

Test Case 11 – Invalid Member ID

Input

```
Member ID: M999  
Book ID: 101
```

Expected Output

```
Member Not Found
```

Test Case 12 – Display Available Books

Input

Assume the library contains:

```
101 Java Programming Available  
102 Python Programming Issued  
103 Database Systems Available
```

Expected Output

Available Books

101 Java Programming
103 Database Systems

Book 102 should not appear because it is currently issued.

Test Case 13 – Add Different Types of Books

Input

101 Java Programming James 500 PrintedBook
102 Database Systems Korth 650 ReferenceBook
103 Python Programming Mark 400 EBook

Expected Output

Book 101 - PrintedBook
Book 102 - ReferenceBook
Book 103 - EBook

The output should demonstrate the appropriate behavior of the different book types.

Test Case 14 – Display Member Details

Input

Member ID: M101

Assume book 101 is issued to the member.

Expected Output

Member ID: M101
Name: Amit
Email: amit@gmail.com

Issued Books:
101 Java Programming

Consolidated Test Case Table

TC	Operation	Input	Expected Result
TC01	Add Book	Valid book details	Book added
TC02	Register Member	Valid member details	Member registered
TC03	Search Book	Existing Book ID	Book details returned
TC04	Search Book	Invalid Book ID	Book Not Found
TC05	Issue Book	Valid member + available book	Book issued
TC06	Check Status	Issued Book ID	Status = Issued
TC07	Issue Book	Already issued book	Book is not available
TC08	Return Book	Valid issued book	Book returned
TC09	Check Status	Returned Book ID	Status = Available
TC10	Return Book	Available book	Invalid return message
TC11	Issue Book	Invalid Member ID	Member Not Found
TC12	Display Books	Library inventory	Only available books displayed
TC13	Book Types	Different book types	Type-specific behavior
TC14	Member Details	Valid Member ID	Member + issued books displayed